JavaScript is disabled on your browser.

**Java SE 27 & JDK 27**

# Class JarFile

java.lang.Object
    java.util.zip.ZipFile
        java.util.jar.JarFile

**All Implemented Interfaces:**

`Closeable`, `AutoCloseable`

---

```
public class JarFile
extends ZipFile
```

The `JarFile` class is used to read the contents of a jar file from any file that can be opened with `java.io.RandomAccessFile`. It extends the class `java.util.zip.ZipFile` with support for reading an optional `Manifest` entry, and support for processing multi-release jar files. The `Manifest` can be used to specify meta-information about the jar file and its entries.

A multi-release jar file is a jar file that contains a manifest with a main attribute named "Multi-Release", a set of "base" entries, some of which are public classes with public or protected methods that comprise the public interface of the jar file, and a set of "versioned" entries contained in subdirectories of the "META-INF/versions" directory. The versioned entries are partitioned by the major version of the Java platform. A versioned entry, with a version `n`, `8 < n`, in the "META-INF/versions/{n}" directory overrides the base entry as well as any entry with a version number `i` where `8 < i < n`.

By default, a `JarFile` for a multi-release jar file is configured to process the multi-release jar file as if it were a plain (unversioned) jar file, and as such an entry name is associated with at most one base entry. The `JarFile` may be configured to process a multi-release jar file by creating the `JarFile` with the `JarFile(File, boolean, int, Runtime.Version)` constructor. The `Runtime.Version` object sets a maximum version used when searching for versioned entries. When so configured, an entry name can correspond with at most one base entry and zero or more versioned entries. A search is required to associate the entry name with the latest versioned entry whose version is less than or equal to the maximum version (see `getEntry(String)`).

Class loaders that utilize `JarFile` to load classes from the contents of `JarFile` entries should construct the `JarFile` by invoking the `JarFile(File, boolean, int, Runtime.Version)` constructor with the value `Runtime.version()` assigned to the last argument. This assures that classes compatible with the major version of the running JVM are loaded from multi-release jar files.

If the `verify` flag is on when opening a signed jar file, the content of the jar entry is verified against the signature embedded inside the manifest that is associated with its `path name`. For a multi-release jar file, the content of a

versioned entry is verified against its own signature and `JarEntry.getCodeSigners()` returns its own signers.

Please note that the verification process does not include validating the signer's certificate. A caller should inspect the return value of `JarEntry.getCodeSigners()` to further determine if the signature can be trusted.

Unless otherwise noted, passing a `null` argument to a constructor or method in this class will cause a `NullPointerException` to be thrown.

**Implementation Note:**

If the API can not be used to configure a `JarFile` (e.g. to override the configuration of a compiled application or library), two `System` properties are available.

- `jdk.util.jar.version` can be assigned a value that is the `String` representation of a non-negative integer <= `Runtime.version().feature()`. The value is used to set the effective runtime version to something other than the default value obtained by evaluating `Runtime.version().feature()`. The effective runtime version is the version that the `JarFile(File, boolean, int, Runtime.Version)` constructor uses when the value of the last argument is `JarFile.runtimeVersion()`.
- `jdk.util.jar.enableMultiRelease` can be assigned one of the three `String` values *true*, *false*, or *force*. The value *true*, the default value, enables multi-release jar file processing. The value *false* disables multi-release jar processing, ignoring the "Multi-Release" manifest attribute, and the versioned directories in a multi-release jar file if they exist. Furthermore, the method `isMultiRelease()` returns *false*. The value *force* causes the `JarFile` to be initialized to runtime versioning
after construction. It effectively does the same as this code: `(new JarFile(File, boolean, int, JarFile.runtimeVersion())`.

**Since:**

1.2

**See Also:**

`Manifest`, `ZipFile`, `JarEntry`

## *Field Summary*

**Fields**

| Modifier and Type | Field | Description |
|---|---|---|
| `static final int` | **`CENATT`** | |

| | | |
|---|---|---|
| | | Central directory (CEN) header internal file attributes field offset. |
| static final int | **CENATX** | Central directory (CEN) header external file attributes field offset. |
| static final int | **CENCOM** | Central directory (CEN) header comment length field offset. |
| static final int | **CENCRC** | Central directory (CEN) header uncompressed file crc-32 value field offset. |
| static final int | **CENDSK** | Central directory (CEN) header disk number start field offset. |
| static final int | **CENEXT** | Central directory (CEN) header extra field length field offset. |
| static final int | **CENFLG** | Central directory (CEN) header encrypt, decrypt flags field offset. |
| static final int | **CENHDR** | Central directory (CEN) header size in bytes (including signature). |
| static final int | **CENHOW** | Central directory (CEN) header compression method field offset. |
| static final int | **CENLEN** | Central directory (CEN) header uncompressed size field offset. |
| static final int | **CENNAM** | Central directory (CEN) header filename length field offset. |
| static final int | **CENOFF** | |

| | | |
|---|---|---|
| | | Central directory (CEN) header LOC header offset field offset. |
| `static final long` | **CENSIG** | Central directory (CEN) header signature. |
| `static final int` | **CENSIZ** | Central directory (CEN) header compressed size field offset. |
| `static final int` | **CENTIM** | Central directory (CEN) header modification time field offset. |
| `static final int` | **CENVEM** | Central directory (CEN) header version made by field offset. |
| `static final int` | **CENVER** | Central directory (CEN) header version needed to extract field offset. |
| `static final int` | **ENDCOM** | End of central directory (END) header ZIP file comment length field offset. |
| `static final int` | **ENDHDR** | End of central directory (END) header size in bytes (including signature). |
| `static final int` | **ENDOFF** | End of central directory (END) header offset for the first CEN header field offset. |
| `static final long` | **ENDSIG** | End of central directory (END) header signature. |
| `static final int` | **ENDSIZ** | End of central directory (END) header central directory size in bytes field offset. |
| `static final int` | **ENDSUB** | |

| | | |
|---|---|---|
| | | End of central directory (END) header number of entries on this disk field offset. |
| static final int | **ENDTOT** | End of central directory (END) header total number of entries field offset. |
| static final int | **EXTCRC** | Extra local (EXT) header uncompressed file crc-32 value field offset. |
| static final int | **EXTHDR** | Extra local (EXT) header size in bytes (including signature). |
| static final int | **EXTLEN** | Extra local (EXT) header uncompressed size field offset. |
| static final long | **EXTSIG** | Extra local (EXT) header signature. |
| static final int | **EXTSIZ** | Extra local (EXT) header compressed size field offset. |
| static final int | **LOCCRC** | Local file (LOC) header uncompressed file crc-32 value field offset. |
| static final int | **LOCEXT** | Local file (LOC) header extra field length field offset. |
| static final int | **LOCFLG** | Local file (LOC) header general purpose bit flag field offset. |
| static final int | **LOCHDR** | Local file (LOC) header size in bytes (including signature). |
| static final int | **LOCHOW** | |

| | | |
|---|---|---|
| | | Local file (LOC) header compression method field offset. |
| static final int | **LOCLEN** | Local file (LOC) header uncompressed size field offset. |
| static final int | **LOCNAM** | Local file (LOC) header filename length field offset. |
| static final long | **LOCSIG** | Local file (LOC) header signature. |
| static final int | **LOCSIZ** | Local file (LOC) header compressed size field offset. |
| static final int | **LOCTIM** | Local file (LOC) header modification time field offset. |
| static final int | **LOCVER** | Local file (LOC) header version needed to extract field offset. |
| static final String | **MANIFEST_NAME** | The JAR manifest file name. |

**Fields declared in class ZipFile**

OPEN_DELETE, OPEN_READ

## *Constructor Summary*

**Constructors**

| Constructor | Description |
|---|---|
| **JarFile**(File file) | Creates a new JarFile to read from the specified File object. |
| **JarFile**(File file, boolean verify) | Creates a new JarFile to read from the specified File object. |

| | |
|---|---|
| **JarFile**(File file, boolean verify, int mode) | Creates a new JarFile to read from the specified File object in the specified mode. |
| **JarFile**(File file, boolean verify, int mode, Runtime.Version version) | Creates a new JarFile to read from the specified File object in the specified mode. |
| **JarFile**(String name) | Creates a new JarFile to read from the specified file name. |
| **JarFile**(String name, boolean verify) | Creates a new JarFile to read from the specified file name. |

## *Method Summary*

All Methods | Static Methods

| Modifier and Type | Method | Description |
|---|---|---|
| static Runtime.Version | **baseVersion**() | Returns the version that represents the unversioned configuration of a multi-release jar file. |
| Enumeration<JarEntry> | **entries**() | Returns an enumeration of the jar file entries. |
| ZipEntry | **getEntry**(String name) | Returns the ZipEntry for the given base entry name or null if not found. |
| InputStream | **getInputStream**(ZipEntry ze) | Returns an input stream for reading the contents of the specified ZIP file entry. |
| JarEntry | **getJarEntry**(String name) | Returns the JarEntry for the given base entry name or null if not found. |
| Manifest | **getManifest**() | |

| | | |
|---|---|---|
| | | Returns the jar file manifest, or `null` if none. |
| final Runtime.Version | **getVersion**() | Returns the maximum version used when searching for versioned entries. |
| final boolean | **isMultiRelease**() | Indicates whether or not this jar file is a multi-release jar file. |
| static Runtime.Version | **runtimeVersion**() | Returns the version that represents the effective runtime versioned configuration of a multi-release jar file. |
| Stream<JarEntry> | **stream**() | Returns an ordered `Stream` over the jar file entries. |
| Stream<JarEntry> | **versionedStream**() | Returns a `Stream` of the versioned jar file entries. |

**Methods declared in class ZipFile**

close, getComment, getName, size, toString

**Methods declared in class Object**

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

## *Field Details*

### MANIFEST_NAME

```
public static final String MANIFEST_NAME
```

The JAR manifest file name.

**See Also:**

Constant Field Values

## LOCSIG

`static final long LOCSIG`

Local file (LOC) header signature.

**See Also:**

Constant Field Values

## EXTSIG

`static final long EXTSIG`

Extra local (EXT) header signature.

**See Also:**

Constant Field Values

## CENSIG

`static final long CENSIG`

Central directory (CEN) header signature.

**See Also:**

Constant Field Values

## ENDSIG

`static final long ENDSIG`

End of central directory (END) header signature.

**See Also:**

Constant Field Values

## LOCHDR

`static final int LOCHDR`

Local file (LOC) header size in bytes (including signature).

**See Also:**

Constant Field Values

## EXTHDR

`static final int EXTHDR`

Extra local (EXT) header size in bytes (including signature).

**See Also:**

Constant Field Values

### CENHDR

`static final int CENHDR`

Central directory (CEN) header size in bytes (including signature).

**See Also:**

Constant Field Values

### ENDHDR

`static final int ENDHDR`

End of central directory (END) header size in bytes (including signature).

**See Also:**

Constant Field Values

### LOCVER

`static final int LOCVER`

Local file (LOC) header version needed to extract field offset.

**See Also:**

Constant Field Values

### LOCFLG

`static final int LOCFLG`

Local file (LOC) header general purpose bit flag field offset.

**See Also:**

Constant Field Values

### LOCHOW

`static final int LOCHOW`

Local file (LOC) header compression method field offset.

**See Also:**

Constant Field Values

### LOCTIM

```
static final int LOCTIM
```

Local file (LOC) header modification time field offset.

**See Also:**

Constant Field Values

### LOCCRC

```
static final int LOCCRC
```

Local file (LOC) header uncompressed file crc-32 value field offset.

**See Also:**

Constant Field Values

### LOCSIZ

```
static final int LOCSIZ
```

Local file (LOC) header compressed size field offset.

**See Also:**

Constant Field Values

### LOCLEN

```
static final int LOCLEN
```

Local file (LOC) header uncompressed size field offset.

**See Also:**

Constant Field Values

### LOCNAM

```
static final int LOCNAM
```

Local file (LOC) header filename length field offset.

**See Also:**

Constant Field Values

### LOCEXT

```
static final int LOCEXT
```

Local file (LOC) header extra field length field offset.

**See Also:**

Constant Field Values

### EXTCRC

`static final int EXTCRC`

Extra local (EXT) header uncompressed file crc-32 value field offset.

**See Also:**

Constant Field Values

### EXTSIZ

`static final int EXTSIZ`

Extra local (EXT) header compressed size field offset.

**See Also:**

Constant Field Values

### EXTLEN

`static final int EXTLEN`

Extra local (EXT) header uncompressed size field offset.

**See Also:**

Constant Field Values

### CENVEM

`static final int CENVEM`

Central directory (CEN) header version made by field offset.

**See Also:**

Constant Field Values

### CENVER

`static final int CENVER`

Central directory (CEN) header version needed to extract field offset.

**See Also:**

Constant Field Values

## CENFLG

```
static final int CENFLG
```

Central directory (CEN) header encrypt, decrypt flags field offset.

**See Also:**

Constant Field Values

## CENHOW

```
static final int CENHOW
```

Central directory (CEN) header compression method field offset.

**See Also:**

Constant Field Values

## CENTIM

```
static final int CENTIM
```

Central directory (CEN) header modification time field offset.

**See Also:**

Constant Field Values

## CENCRC

```
static final int CENCRC
```

Central directory (CEN) header uncompressed file crc-32 value field offset.

**See Also:**

Constant Field Values

## CENSIZ

```
static final int CENSIZ
```

Central directory (CEN) header compressed size field offset.

**See Also:**

Constant Field Values

## CENLEN

```
static final int CENLEN
```

Central directory (CEN) header uncompressed size field offset.

**See Also:**

Constant Field Values

### CENNAM

`static final int CENNAM`

Central directory (CEN) header filename length field offset.

**See Also:**

Constant Field Values

### CENEXT

`static final int CENEXT`

Central directory (CEN) header extra field length field offset.

**See Also:**

Constant Field Values

### CENCOM

`static final int CENCOM`

Central directory (CEN) header comment length field offset.

**See Also:**

Constant Field Values

### CENDSK

`static final int CENDSK`

Central directory (CEN) header disk number start field offset.

**See Also:**

Constant Field Values

### CENATT

`static final int CENATT`

Central directory (CEN) header internal file attributes field offset.

**See Also:**

Constant Field Values

## CENATX

```
static final int CENATX
```

Central directory (CEN) header external file attributes field offset.

**See Also:**

Constant Field Values

## CENOFF

```
static final int CENOFF
```

Central directory (CEN) header LOC header offset field offset.

**See Also:**

Constant Field Values

## ENDSUB

```
static final int ENDSUB
```

End of central directory (END) header number of entries on this disk field offset.

**See Also:**

Constant Field Values

## ENDTOT

```
static final int ENDTOT
```

End of central directory (END) header total number of entries field offset.

**See Also:**

Constant Field Values

## ENDSIZ

```
static final int ENDSIZ
```

End of central directory (END) header central directory size in bytes field offset.

**See Also:**

Constant Field Values

## ENDOFF

```
static final int ENDOFF
```

End of central directory (END) header offset for the first CEN header field offset.

**See Also:**

Constant Field Values

### ENDCOM

```
static final int ENDCOM
```

End of central directory (END) header ZIP file comment length field offset.

**See Also:**

Constant Field Values

## *Constructor Details*

### JarFile

```
public JarFile(String name)
        throws IOException
```

Creates a new `JarFile` to read from the specified file `name`. The `JarFile` will be verified if it is signed.

**Parameters:**

`name` - the name of the jar file to be opened for reading

**Throws:**

`IOException` - if an I/O error has occurred

### JarFile

```
public JarFile(String name,
               boolean verify)
        throws IOException
```

Creates a new `JarFile` to read from the specified file `name`.

**Parameters:**

`name` - the name of the jar file to be opened for reading

`verify` - whether or not to verify the jar file if it is signed.

**Throws:**

`IOException` - if an I/O error has occurred

## JarFile

```
public JarFile(File file)
        throws IOException
```

Creates a new `JarFile` to read from the specified `File` object. The `JarFile` will be verified if it is signed.

**Parameters:**

`file` - the jar file to be opened for reading

**Throws:**

`IOException` - if an I/O error has occurred

## JarFile

```
public JarFile(File file,
               boolean verify)
        throws IOException
```

Creates a new `JarFile` to read from the specified `File` object.

**Parameters:**

`file` - the jar file to be opened for reading

`verify` - whether or not to verify the jar file if it is signed.

**Throws:**

`IOException` - if an I/O error has occurred

## JarFile

```
public JarFile(File file,
               boolean verify,
               int mode)
        throws IOException
```

Creates a new `JarFile` to read from the specified `File` object in the specified mode. The mode argument must be either `OPEN_READ` or `OPEN_READ | OPEN_DELETE`.

**Parameters:**

`file` - the jar file to be opened for reading

`verify` - whether or not to verify the jar file if it is signed.

`mode` - the mode in which the file is to be opened

**Throws:**

`IOException` - if an I/O error has occurred

`IllegalArgumentException` - if the `mode` argument is invalid

**Since:**

1.3

### JarFile

```
public JarFile(File file,
               boolean verify,
               int mode,
               Runtime.Version version)
        throws IOException
```

Creates a new `JarFile` to read from the specified `File` object in the specified mode. The mode argument must be either `OPEN_READ` or `OPEN_READ | OPEN_DELETE`. The version argument, after being converted to a canonical form, is used to configure the `JarFile` for processing multi-release jar files.

The canonical form derived from the version parameter is `Runtime.Version.parse(Integer.toString(n))` where `n` is `Math.max(version.feature(), JarFile.baseVersion().feature())`.

**Parameters:**

`file` - the jar file to be opened for reading

`verify` - whether or not to verify the jar file if it is signed.

`mode` - the mode in which the file is to be opened

`version` - specifies the release version for a multi-release jar file

**Throws:**

`IOException` - if an I/O error has occurred

`IllegalArgumentException` - if the `mode` argument is invalid

`NullPointerException` - if `version` is `null`

**Since:**

9

## *Method Details*

### baseVersion

```
public static Runtime.Version baseVersion()
```

Returns the version that represents the unversioned configuration of a multi-release jar file.

**Returns:**

the version that represents the unversioned configuration

**Since:**

9

## runtimeVersion

`public static Runtime.Version runtimeVersion()`

Returns the version that represents the effective runtime versioned configuration of a multi-release jar file.

By default the feature version number of the returned `Version` will be equal to the feature version number of `Runtime.version()`. However, if the `jdk.util.jar.version` property is set, the returned `Version` is derived from that property and feature version numbers may not be equal.

**Returns:**

the version that represents the runtime versioned configuration

**Since:**

9

## getVersion

`public final Runtime.Version getVersion()`

Returns the maximum version used when searching for versioned entries.

If this `JarFile` is not a multi-release jar file or is not configured to be processed as such, then the version returned will be the same as that returned from `baseVersion()`.

**Returns:**

the maximum version

**Since:**

9

## isMultiRelease

`public final boolean isMultiRelease()`

Indicates whether or not this jar file is a multi-release jar file.

**Returns:**

true if this JarFile is a multi-release jar file

**Since:**

9

## getManifest

```
public Manifest getManifest()
                     throws IOException
```

Returns the jar file manifest, or `null` if none.

**Returns:**

the jar file manifest, or `null` if none

**Throws:**

`IllegalStateException` - may be thrown if the jar file has been closed

`IOException` - if an I/O error has occurred

## getJarEntry

```
public JarEntry getJarEntry(String name)
```

Returns the `JarEntry` for the given base entry name or `null` if not found.

If this `JarFile` is a multi-release jar file and is configured to be processed as such, then a search is performed to find and return a `JarEntry` that is the latest versioned entry associated with the given entry name. The returned `JarEntry` is the versioned entry corresponding to the given base entry name prefixed with the string `"META-INF/versions/{n}/"`, for the largest value of `n` for which an entry exists. If such a versioned entry does not exist, then the `JarEntry` for the base entry is returned, otherwise `null` is returned if no entries are found. The initial value for the version `n` is the maximum version as returned by the method `getVersion()`.

**Implementation Requirements:**

This implementation invokes `getEntry(String)`.

**Parameters:**

`name` - the jar file entry name

**Returns:**

the `JarEntry` for the given entry name, or the versioned entry name, or `null` if not found

**Throws:**

`IllegalStateException` - may be thrown if the jar file has been closed

**See Also:**

`JarEntry`

## getEntry

```
public ZipEntry getEntry(String name)
```

Returns the `ZipEntry` for the given base entry name or `null` if not found.

If this `JarFile` is a multi-release jar file and is configured to be processed as such, then a search is performed to find and return a `ZipEntry` that is the latest versioned entry associated with the given entry name. The returned `ZipEntry` is the versioned entry corresponding to the given base entry name prefixed with the string `"META-INF/versions/{n}/"`, for the largest value of `n` for which an entry exists. If such a versioned entry does not exist, then the `ZipEntry` for the base entry is returned, otherwise `null` is returned if no entries are found. The initial value for the version `n` is the maximum version as returned by the method `getVersion()`.

**Overrides:**

`getEntry` in class `ZipFile`

**Implementation Requirements:**

This implementation may return a versioned entry for the requested name even if there is not a corresponding base entry. This can occur if there is a private or package-private versioned entry that matches. If a subclass overrides this method, assure that the override method invokes `super.getEntry(name)` to obtain all versioned entries.

**Parameters:**

`name` - the jar file entry name

**Returns:**

the `ZipEntry` for the given entry name or the versioned entry name or `null` if not found

**Throws:**

`IllegalStateException` - may be thrown if the jar file has been closed

**See Also:**

`ZipEntry`

## entries

```
public Enumeration<JarEntry> entries()
```

Returns an enumeration of the jar file entries.

**Overrides:**

`entries` in class `ZipFile`

**Returns:**

an enumeration of the jar file entries

**Throws:**

`IllegalStateException` - may be thrown if the jar file has been closed

### stream

```
public Stream<JarEntry> stream()
```

Returns an ordered `Stream` over the jar file entries. Entries appear in the `Stream` in the order they appear in the central directory of the jar file.

**Overrides:**

`stream` in class `ZipFile`

**Returns:**

an ordered `Stream` of entries in this jar file

**Throws:**

`IllegalStateException` - if the jar file has been closed

**Since:**

1.8

### versionedStream

```
public Stream<JarEntry> versionedStream()
```

Returns a `Stream` of the versioned jar file entries.

If this `JarFile` is a multi-release jar file and is configured to be processed as such, then an entry in the stream is the latest versioned entry associated with the corresponding base entry name. The maximum version of the latest versioned entry is the version returned by `getVersion()`. The returned stream may include an entry that only exists as a versioned entry. If the jar file is not a multi-release jar file or the `JarFile` is not configured for processing a multi-release jar file, this method returns the same stream that `stream()` returns.

**Returns:**

stream of versioned entries

**Since:**

10

### getInputStream

```
public InputStream getInputStream(ZipEntry ze)
                           throws IOException
```

Returns an input stream for reading the contents of the specified ZIP file entry.

**Overrides:**

`getInputStream` in class `ZipFile`

**API Note:**

The `InputStream` returned by this method can wrap an `InflaterInputStream`, whose `read(byte[], int, int)` method can modify any element of the output buffer.

**Parameters:**

ze - the ZIP file entry

**Returns:**

an input stream for reading the contents of the specified ZIP file entry or null if the ZIP file entry does not exist within the jar file

**Throws:**

`ZipException` - if a ZIP file format error has occurred

`IOException` - if an I/O error has occurred

`SecurityException` - if any of the jar file entries are incorrectly signed.

`IllegalStateException` - may be thrown if the jar file has been closed

---

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.